Sprint 2 Plan

Vitae | Jan 28 – Feb 10, 2026 | Revision #1 | Last Modified: 2026-02-10

Goals

Users should be able to build, save, and preview resumes based on their career history.

User Stories + Task Assignments

1. Select Items for Resume (2 points): As a job applicant, I can select which career history items appear in my current resume so that I can tailor content for a specific job opportunity.

    1.1. Design working_state schema (1 hour) Alex Lio
    1.2. Create working_state table in DB (1 hour) Alex Lio
    1.3. Create API endpoint: GET /working-state (1 hour) Alex Lio
    1.4. Create API endpoint: PUT /working-state (2 hours) Alex Lio
    1.5. Design resume item selection UI with select/deselect toggles (3 hours) Omer Boztepe
    1.6. Implement select/deselect functionality (2 hours) Omer Boztepe
    1.7. Implement auto-save on selection change (with debouncing) (2 hours) Omer Boztepe
    1.8. Show selected items in resume preview pane (no PDF yet) (2 hours) Omer Boztepe
    1.9. Write unit tests for PUT /working-state validation (1 hour) Omer Boztepe
    1.10. Write integration and/or unit test(s)/manual tests for select/deselect flow (2 hours) Omer Boztepe

Total hours for user story 1: 17 hours

---

2. Resume Reordering (3 points): As a user, I can reorder sections and items within my resume so that I can prioritize what matters most for an application or audience.
    2.1. Add section order and item order to working state schema (1 hour) Keith Tran
    2.2. Add drag-and-drop library for reordering in the UI (2 hours) Keith Tran

2.3. Implement section reordering (3 hours) Keith Tran
2.4. Implement item reordering within a section (3 hours) Keith Tran
2.5. Auto-save order changes to working state (2 hours) Keith Tran
2.6. Handle edge cases (empty section, single item, etc) (1 hour) Keith Tran
2.7. Write unit tests for order validation logic (1 hour) Keith Tran
2.8. Write integration and/or unit test(s)/manual tests for reorder persistence (2 hours) Keith Tran

Total hours for user story 2: 15 hours

---

3. Save Named Resumes (4 points): As a user, I can name and save my current resume so that I can retrieve it later.
   3.1. Design versions table schema (1 hour) Alex Lio
   3.2. Create versions table in DB (1 hour) Alex Lio
   3.3. Create API endpoint: POST /versions (2 hours) Berat Ercevik
   3.4. Design “Save Resume” UI with name input (2 hours) Berat Ercevik
   3.5. Implement name validation (non-empty, unique for the user) (1 hour) Berat Ercevik
   3.6. Connect Save Resume UI to backend (2 hours) Berat Ercevik
   3.7. Show successful save confirmation in UI (2 hours) Berat Ercevik
   3.8. Write unit tests for saved resume creation logic (2 hours) Berat Ercevik
   3.9. Write unit tests for name uniqueness validation (1 hour) Berat Ercevik
   3.10. Write integration and/or unit test(s)/manual tests for full save flow (2 hours) Berat Ercevik

Total hours for user story 3: 16 hours

---

4. View Named Resumes (2 points): As a user, I can view my saved resumes so that I can look at past versions.
   4.1. Create API endpoint: GET /versions (1 hour) Omer Boztepe
   4.2. Design saved resumes list / visualization UI (3 hours) Omer Boztepe
   4.3. Implement saved resumes visualization (2 hours) Omer Boztepe
   4.4. Display version metadata (name, datetime created) (1 hour) Omer Boztepe
   4.5. Handle empty state (no saved resumes yet) (1 hour) Omer Boztepe
   4.6. Write unit tests for GET /versions (filters by user, sorts by date) (1 hour) Omer Boztepe
   4.7. Write integration and/or unit test(s)/manual tests for list retrieval (1 hour) Omer Boztepe

Total hours for user story 4: 10 hours

---

5. Restore Named Resume (3 points): As a user, I can restore a saved resume to my working state so that I can continue editing or reuse it.
    5.1. Create API endpoint: POST /versions/:id/restore (which restores version to working state) (2 hours) Alex Lio
    5.2. Add “Restore” button to each named version in the list / visualization (1 hour) Berat Ercevik
    5.3. Implement confirmation modal - “This will replace your current draft” (2 hours) Berat Ercevik
    5.4. Redirect to resume editor after restore (1 hour) Berat Ercevik
    5.5. Connect restore button to API in backend (2 hours) Alex Lio
    5.6. Handle error states: version not found, network failure, etc (1 hour) Alex Lio
    5.7. Write unit tests for restore logic (make sure snapshot overrides working state) (2 hours) Berat Ercevik
    5.8. Write unit tests for authorization (can’t restore another user’s version) (1 hour) Berat Ercevik
    5.9. Write integration and/or unit test(s)/manual tests for full restore flow (2 hours) Berat Ercevik

Total hours for user story 5: 14 hours

---

6. Live preview (6 points): As a user, I can see a live preview of my resume as a formatted PDF so that I know what the final output looks like.
    6.1. Research Typst rendering in browser (2 hours) Njeri Gachoka
    6.2. Set up Typst compilation pipeline (4 hours) Njeri Gachoka
    6.3. Create / pick basic Typst resume template (2 hours) Njeri Gachoka
    6.4. Write working state JSON -> Typst render function (3 hours) Njeri Gachoka
    6.5. Integrate PDF viewer in resume editor (2 hours) Njeri Gachoka
    6.6. Wire up preview to compile on working state changes (debounced) (2 hours) Njeri Gachoka
    6.7. Handle compilation errors (2 hours) Njeri Gachoka
    6.8. Performance optimizations (don’t rerender on every keystroke) (2 hours) Njeri Gachoka
    6.9. Write integration and/or unit test(s)/manual tests for JSON -> Typst conversion (2 hours) Njeri Gachoka
    6.10. Write integration and/or unit test(s)/manual tests for error handling (malformed input) (1 hour) Njeri Gachoka

Total hours for user story 6: 22 hours

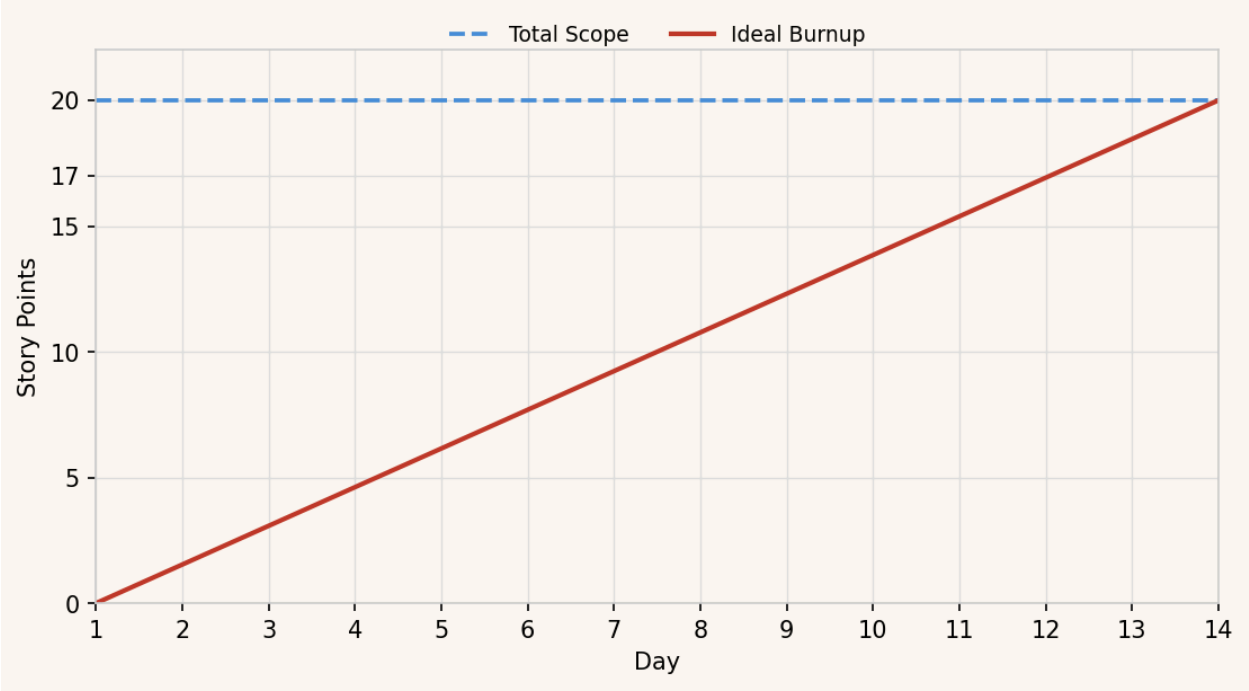
Role Assignments

Njeri Gachoka : Developer, Product Owner
Keith Tran : Developer
Alex Lio : Developer
Berat Ercevik : Developer, Scrum Master
Omer Boztepe : Developer

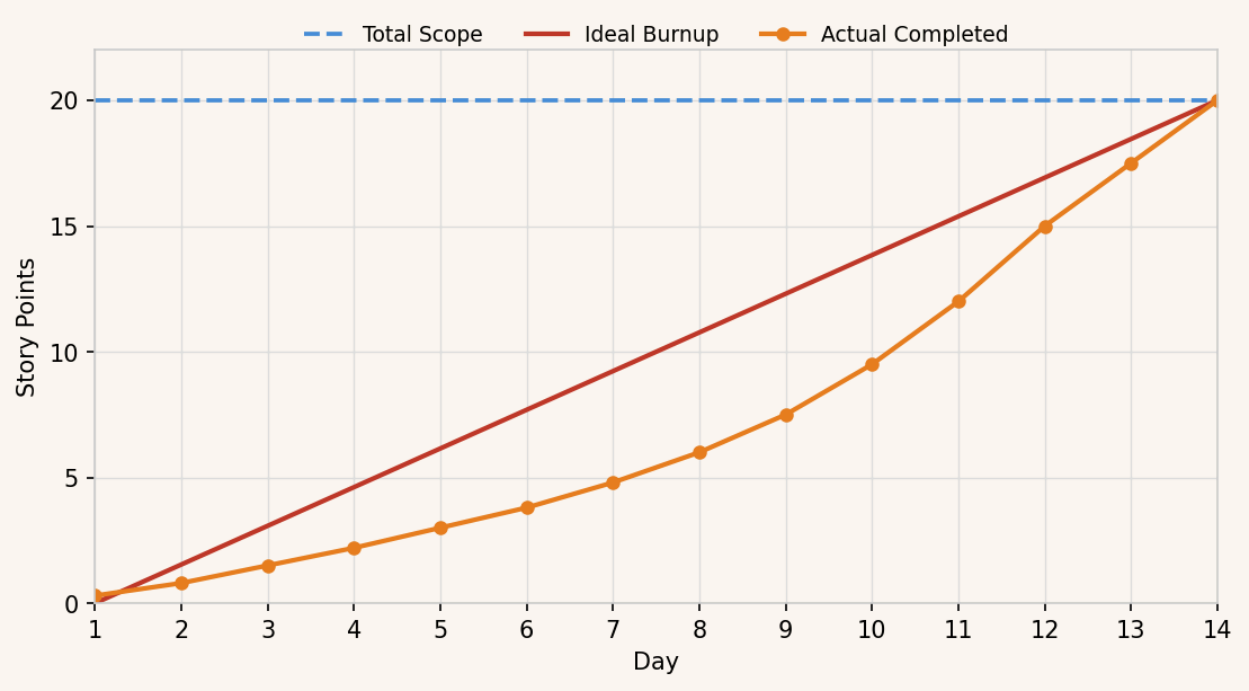
Initial Task Assignments

See user story and task listings above

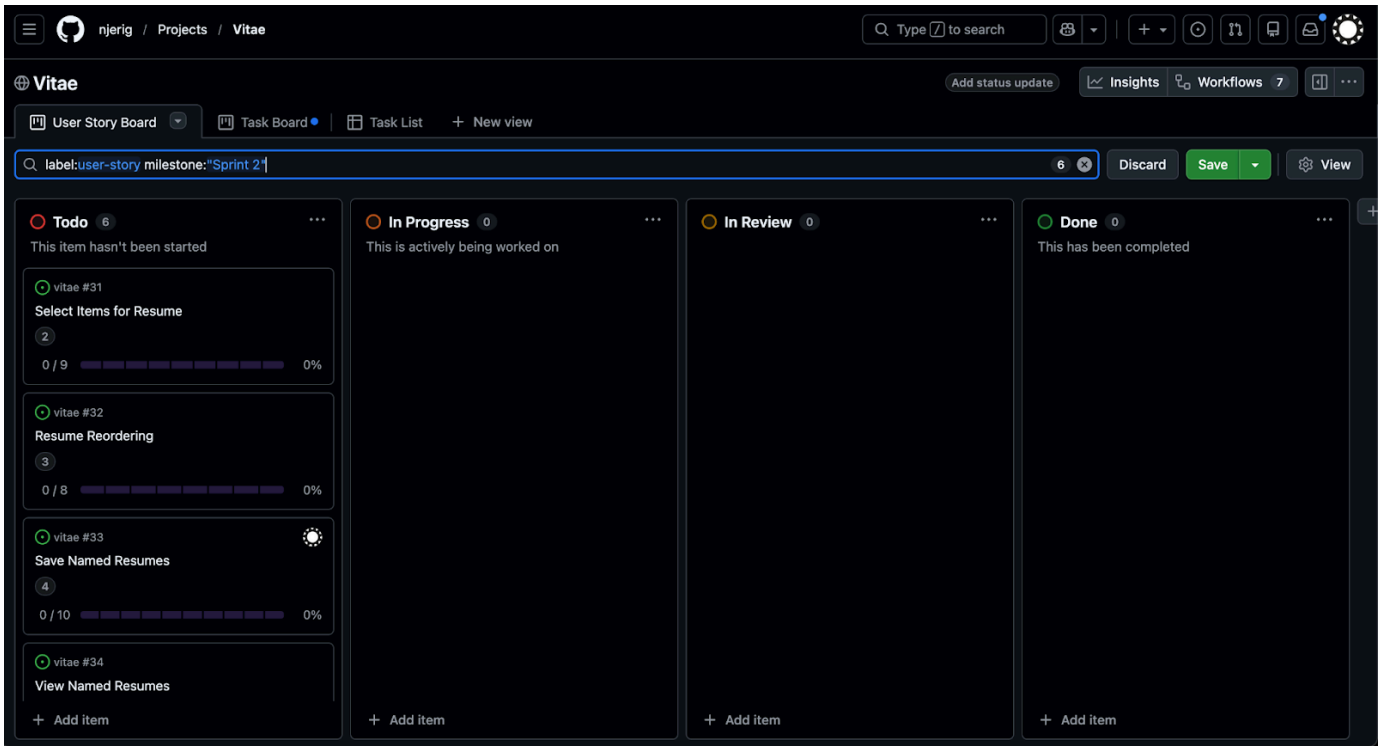
Initial Burnup Chart



Final Burnup Chart



Scrum Board



Scrum Times

Friday 12:00pm - 1:00pm (TA meeting with Michelle Quach)
Friday 1:00pm - 1:15pm
Tuesday 10:00am - 10:15am
Sunday 10:00am - 10:15am
Monday 7:00pm - 9:00pm (Optional) general work-together hours